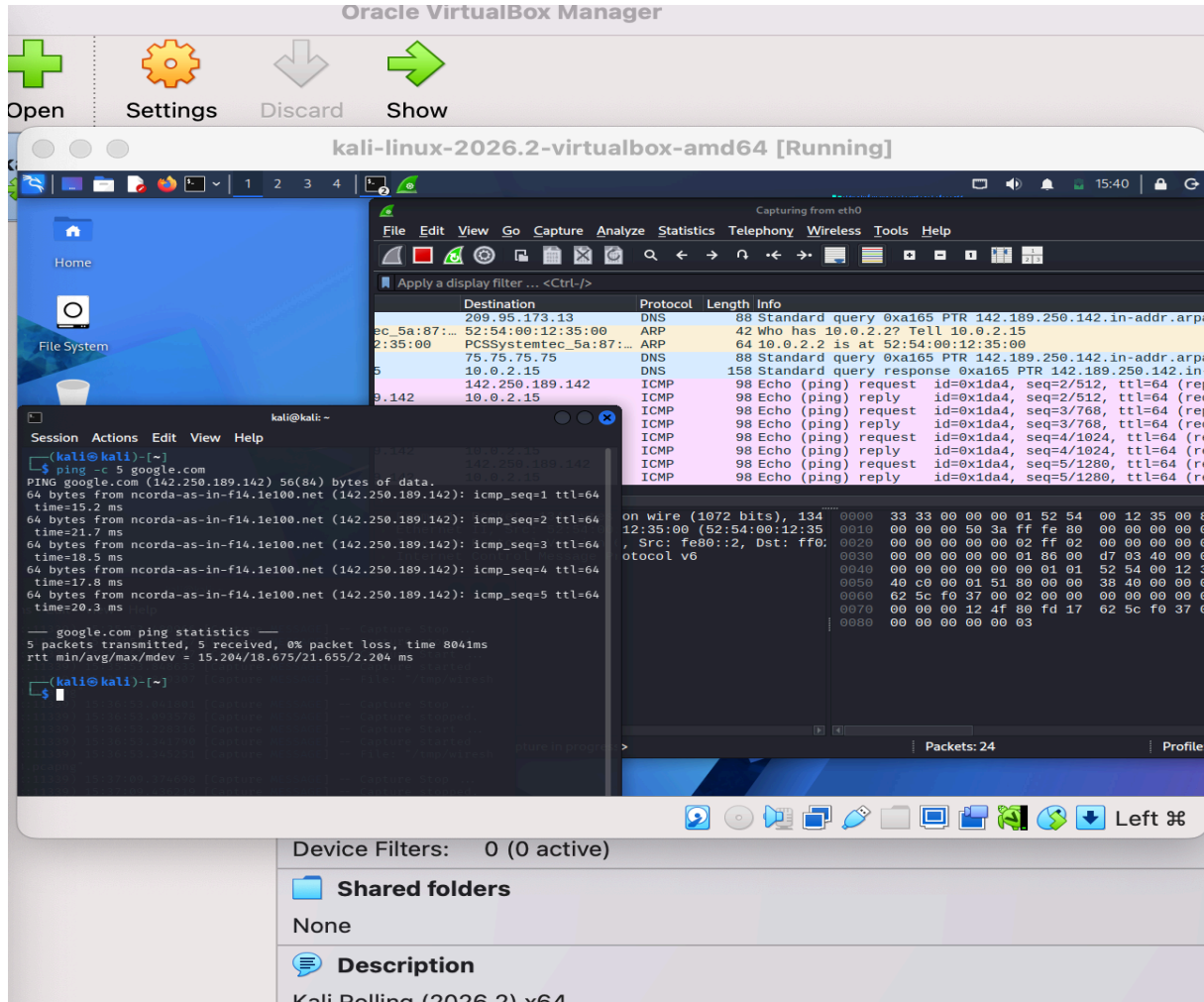


Home SOC Lab Documentation - Syedda Aqsa

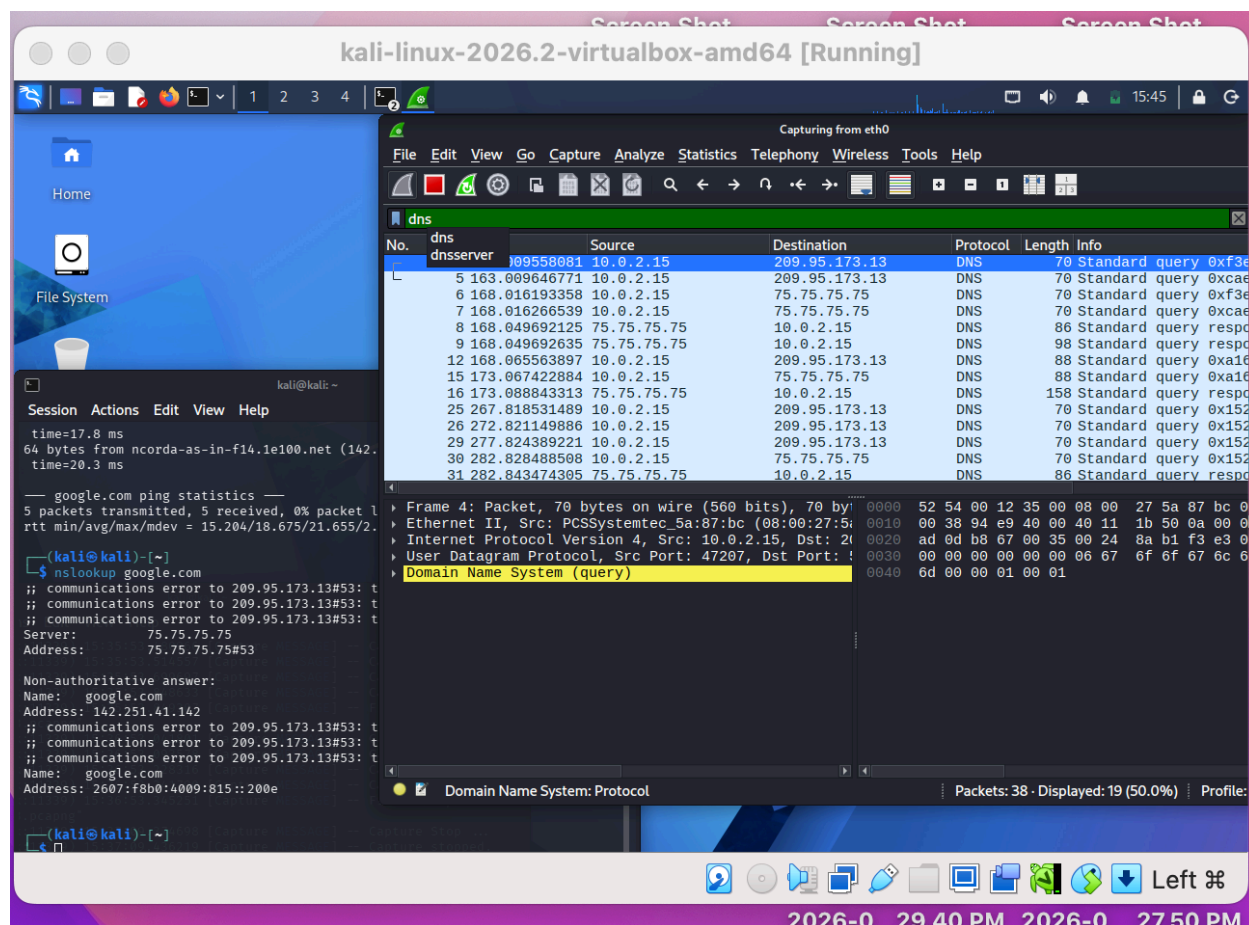


Initial Setup & ICMP Test

What I did: Set up a Kali Linux 2026.2 VM in VirtualBox with a working network connection, then opened Wireshark and started a live capture on the `eth0` interface. From the terminal, I ran `ping -c 5 google.com` to test basic outbound connectivity.

What the screenshot shows: The terminal output confirms 5 packets were sent and received with 0% packet loss, with round-trip times between ~15–20ms. In Wireshark, I can see the corresponding traffic: a DNS query resolving `google.com`, followed by a burst of ICMP echo request/reply pairs, plus a PTR (reverse DNS) lookup on the resolved IP.

Why it matters: This confirms the full chain of events behind a simple `ping`: the OS first resolves the hostname via DNS, then sends ICMP echo requests to the resolved IP, and separately performs a reverse DNS lookup on that IP. Seeing this in Wireshark (rather than just trusting the terminal output) proves I can correlate CLI behavior with what's actually happening on the wire.

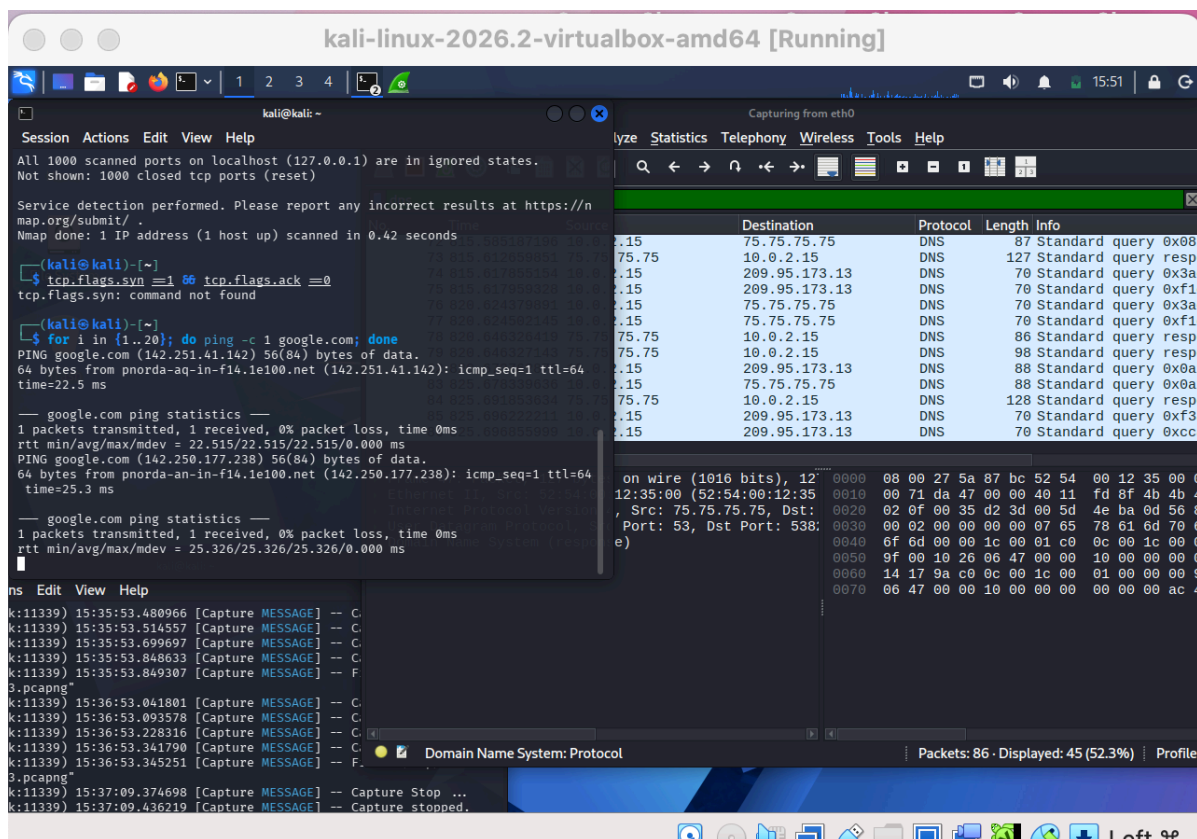


DNS Query Analysis

What I did: Ran `nslookup google.com` directly to manually trigger and inspect a DNS resolution, then filtered the Wireshark capture down to `dns` traffic only to isolate those packets from the rest of the noise.

What the screenshot shows: The `nslookup` output shows the query going through my configured DNS server, returning both an IPv4 and IPv6 address for `google.com`. The filtered Wireshark view lines up with this, showing repeated standard DNS queries and responses between my VM (`10.0.2.15`) and the resolver (`75.75.75.75`), along with the packet's raw hex/ASCII breakdown at the bottom (Ethernet, IP, and UDP headers).

Why it matters: This shows I understand DNS isn't a single request/response, a host can send multiple queries (A, AAAA, PTR) for one lookup, and knowing how to filter Wireshark down to just `dns` traffic is a basic but essential skill for cutting through noise during real investigations.

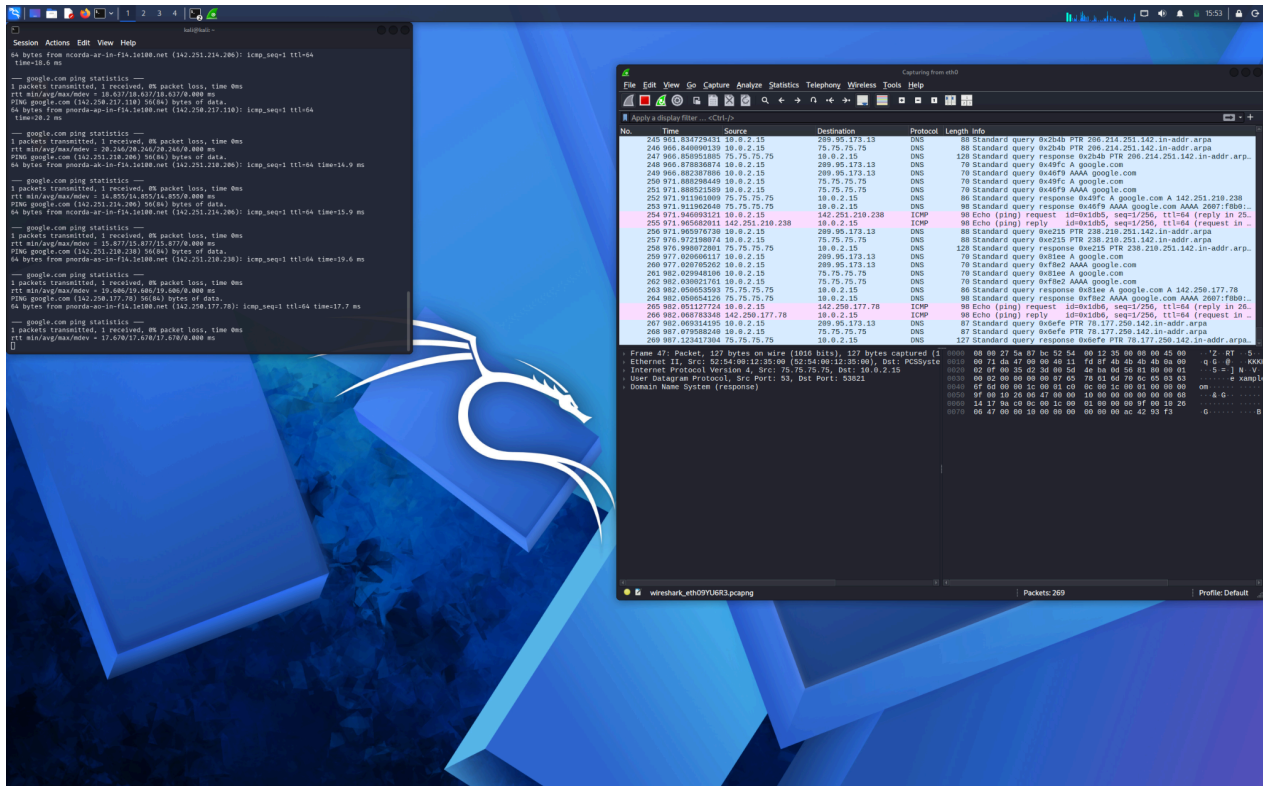


Port Scanning & Automated Traffic Generation

What I did: Ran an `nmap` scan against `localhost` to enumerate open ports and running services on my own VM. I also used a simple bash loop (`for i in {1..20}; do ping -c 1 google.com; done`) to generate a batch of repeated ICMP traffic for a larger sample to review in Wireshark, and saved that capture session to a `.pcapng` file.

What the screenshot shows: The `nmap` output confirms all 1000 scanned ports on localhost came back closed (expected, since no services are exposed), with the scan completing in under a second. Below that, the loop output shows 20 individual ping attempts each with their own stats, and the capture log confirms the Wireshark session was actively recording and later stopped/saved.

Why it matters: This demonstrates two separate but related skills: using Nmap for basic host/port discovery, and generating controlled traffic samples to analyze rather than relying on random background noise. Saving the capture as `.pcapng` also shows I know how to preserve evidence for later or offline analysis, which mirrors how a real SOC analyst would archive captures for a case file.



Full Capture Review

What I did: Reviewed the complete unfiltered capture after stopping the recording, looking at the mix of ARP, DNS, and ICMP traffic generated across the session, including one packet's full protocol breakdown down to the hex level.

What the screenshot shows: The packet list shows a realistic mix of traffic types — ARP requests/replies (used for local network address resolution), DNS queries/responses, and ICMP echo request/reply pairs — all interleaved in the order they actually occurred. The detail pane for the selected packet breaks down the Ethernet frame, IP header, and UDP header field by field.

Why it matters: This is the "big picture" view, being able to look at an unfiltered capture and mentally sort traffic by protocol and purpose is a core SOC analyst skill. It's also the foundation for later work like spotting anomalies (e.g., unexpected ARP traffic, DNS to a suspicious domain, or ICMP used for tunneling) once I start layering in intentionally malicious traffic in future lab iterations.